Documentation for 31st Jan 2025 (.NET)

(9am-10am)

Structure of an ASP.NET Core Project

In an ASP.NET Core application, the project follows a standard structure that separates concerns and helps maintain scalability and organization. Below is an overview of the typical structure, with an explanation of each component.

---

1. Controllers

Location: /Controllers folder
Purpose: Controllers are responsible for handling HTTP requests and returning appropriate responses. They act as the intermediaries between the Model and View layers.

- A Controller typically has action methods that handle different HTTP verbs like GET, POST, PUT, DELETE.
- They usually interact with Models to fetch or update data, then return the result (often to Views or as JSON data for Web APIs).

Example:

```
public class HomeController : Controller
{
  public IActionResult Index()
  {
    return View();
  }
}
```

---

2. Views

Location: /Views folder
Purpose: Views contain the HTML and Razor syntax to render dynamic content. They are responsible for presenting data to the user.

- Views are used to display the output returned by the controller's action methods.

- They can contain both HTML and C# code using Razor syntax to generate dynamic content.
- A view corresponds to a controller's action method (e.g., Index.cshtml corresponds to Index() in HomeController).

Example:

```
@{
   ViewData["Title"] = "Home Page";
}

<h2>@ViewData["Title"]</h2>
```

(10am-11am)

3. Models

Location: /Models folder
Purpose: Models represent the data structure and business logic of the application. They define how data is passed between the application layers.

- A Model can represent data objects (like a Product or Customer) or serve as Data Transfer Objects (DTOs).
- Models also represent how data is handled or validated, often used with Entity Framework (EF) for database operations.

Example:

```
public class Product
{
   public int Id { get; set; }
   public string Name { get; set; }
   public decimal Price { get; set; }
}
```

---

4. appsettings.json

Location: Root of the project
Purpose: The appsettings.json file is used to store configuration settings in a key-value pair format. It's commonly used for app-wide settings, like database connections, third-party service configurations, or environment-specific values.

- appsettings.json is typically used for storing non-sensitive settings, while sensitive settings like API keys should be stored in appsettings.Development.json or environment variables.
- You can have environment-specific configurations, such as appsettings.Development.json for development and appsettings.Production.json for production.

Example:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=MyDb;Trusted_Connection=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning"
    }
  }
}
```

(11am-12pm)

5. Program.cs

Location: Root of the project
Purpose: Program.cs is the entry point for the application. It configures and starts the ASP.NET Core host, sets up dependency injection, and defines middleware pipelines.

- In ASP.NET Core 6 and beyond, this file combines the functionality that was previously in Startup.cs (in older versions).

- It contains the CreateHostBuilder method, which sets up the host and the app's configuration.

Example:

```
public class Program
{
  public static void Main(string[] args)
  {
    CreateHostBuilder(args).Build().Run();
  }

  public static IHostBuilder CreateHostBuilder(string[] args) =>
    Host.CreateDefaultBuilder(args)
      .ConfigureWebHostDefaults(webBuilder =>
      {
        webBuilder.UseStartup<Startup>();
      });
}
```

---

6. Startup.cs

Location: Root of the project
Purpose: In ASP.NET Core versions prior to 6, Startup.cs was the main file to configure services and middleware. While it's now combined into Program.cs for ASP.NET Core 6+, the concept of configuring services and middleware is still very much alive.

- It contains methods like ConfigureServices() (for adding services to the dependency injection container) and Configure() (for defining the request pipeline via middleware).
- Even though Startup.cs is merged in newer versions, understanding the separation between services and middleware configuration remains important.

Example:

```
public class Startup
```

```
{
    public void ConfigureServices(IServiceCollection services)
    {
        services.AddControllersWithViews();
    }

    public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
    {
        if (env.IsDevelopment())
        {
            app.UseDeveloperExceptionPage();
        }
        else
        {
            app.UseExceptionHandler("/Home/Error");
            app.UseHsts();
        }

        app.UseHttpsRedirection();
        app.UseStaticFiles();
        app.UseRouting();
        app.UseAuthorization();

        app.UseEndpoints(endpoints =>
        {
            endpoints.MapControllerRoute(
                name: "default",
                pattern: "{controller=Home}/{action=Index}/{id?}");
```

```
    });
  }
}
```

(12pm-1pm)

7. wwwroot Folder

Location: Root of the project
Purpose: The wwwroot folder is the default location for static files in ASP.NET Core, such as JavaScript files, CSS files, images, and other resources.

- Files placed in wwwroot are publicly accessible and can be referenced directly in Views or through URLs.
- It is important not to place sensitive or non-static files here, as they will be publicly available.

Example:

```
/wwwroot
  /css
    style.css
  /js
    script.js
  /images
    logo.png
```

Putting It All Together: A Typical Project Structure

Here's how the structure would look in an ASP.NET Core MVC project:

```
/MyApp
  /Controllers
    HomeController.cs
  /Models
    Product.cs
```

```
/Views
  /Home
    Index.cshtml
/wwwroot
  /css
    style.css
  /js
    script.js
/appsettings.json
Program.cs
Startup.cs
```

Understanding the components of an ASP.NET Core project helps to stay organized and ensures that each part of the application serves a clear and separate purpose. The MVC architecture—Model, View, Controller—encourages a clean separation of concerns and makes it easier to scale and maintain your application.